

Отчет Костылевой Э. П.
Лабораторная работа №3
Численные методы решения нелинейных уравнений

Цель: Разработать программный код для отделения корней нелинейных уравнений; разработать программный код для нахождения корней нелинейных уравнений методом хорд, методом касательных, методом деления отрезков. Провести сравнительный анализ результатов.

Используемое оборудование: ПК, Microsoft Office, PyCharm.

Часть 1.

Постановка задачи: Разработать программный модуль, который выполняет отделение корней.

Фрагмент кода для отделения корней:

```
850      #Код окна для отделения корней нелинейного уравнения
851      def otdel_korney():
852          def l_gran(C):
853              i = -1
854              while nu_func(i) > C:
855                  i *= 2
856              return i
857          def p_gran(C):
858              i = 1
859              while nu_func(i) < C:
860                  i *= 2
861              return i
862          def monoton(a, b, step = 1e-6):
863              x = a
864              sign = np.sign(nu_proixv_func(x))
865              while x < b:
866                  if sign == np.sign(nu_proixv_func(x + step)):
867                      x += step
868                  else:
869                      return False
870              return True
```

```
871      def ot_kor(a, b):
872          while True:
873              mon = monoton(a, b, step=1e-6)
874              if (nu_func(a) * nu_func(b) < 0) and (mon == True):
875                  return a, b
876              else:
877                  mid = (b - a) * 0.5
878                  if nu_func(mid) < 0:
879                      a = mid
880                  else:
881                      b = mid
882
883      return ot_kor(l_gran(0), p_gran(0))
```

Результат:

Уравнение:

$$x^3 + 2x - 6 = 0$$

Отрезок:

Отрезок (-1, 2)

Часть 2.

Постановка задачи: Разработать программный модуль, который находит корень нелинейного уравнения методом хорд.

Фрагмент кода для метода хорд:

```
771 #Код окна решения нелинейных уравнений методом хорд
772 def resh_nu_met_chord():
773     e = float(dane.get())
774     n = float(dann.get())
775     a, b = resh_otdel_korney()
776     iter = 0
777     while abs(nu_func(b)) > e and iter < n:
778         c = (a * nu_func(b) - b * nu_func(a)) / (nu_func(b) - nu_func(a))
779         if nu_func(c) * nu_func(a) < 0:
780             b = c
781         else: a = c
782         iter += 1
783     print(f'Корень нелинейного уравнения с точностью {e} и {n} разбиений равен {c}')
```

Часть 3.

Постановка задачи: Разработать программный модуль, который находит корень нелинейного уравнения методом касательных.

Фрагмент кода для метода касательных:

```

836 #Код окна решения нелинейных уравнений методом касательных
837 def resh_nu_met_kasat():
838     e = float(dane.get())
839     a, b = resh_otdel_korney()
840     if nu_func(a) * nu_func(b) > 0:
841         x = a
842     else: x = b
843     print(f'При начальном значении x = {x}')
844     x1 = x - nu_func(x) / nu_proixv_func(x)
845     while abs(x1-x) >= e:
846         x = x1 - nu_func(x1) / nu_proixv_func(x1)
847         x1 = x - nu_func(x) / nu_proixv_func(x)
848     print(f'Корень нелинейного уравнения с точностью {e} равен {x}')
849

```

Часть 4.

Постановка задачи: Разработать программный модуль, который находит корень нелинейного уравнения методом деления отрезков.

Фрагмент кода для метода деления отрезков:

```

805 #Код окна решения нелинейных уравнений методом деления отрезков
806 def resh_nu_met_del_otr():
807     e = float(dane.get())
808     a, b = resh_otdel_korney()
809     while (b-a) > e:
810         del_pol = (a+b)/2
811         if nu_func(a) * nu_func(del_pol) < 0:
812             b = del_pol
813         else: a = del_pol
814     print(f'Корень нелинейного уравнения с точностью {e} равен {(a+b)/2}')
815

```

Таблица результатов:

Уравнение:

$$x^3 + 2x - 6 = 0$$

Отрезок (-1, 2)

Метод	e=0,001	e = 0,0001	e=0,00001	e=0,000001	Точное
-------	---------	------------	-----------	------------	--------

					значение
Хорд	1.456164	1.456164	1.456164	1.456164	1.456164
Касательных	1.456164	1.456164	1.456164	1.456164	1.456164
Деления отрезков	1.456176	1.456130	1.456162	1.456164	1.456164

Файл с программой: <https://disk.yandex.ru/d/QxrSTxIwro7Vcg>

Ссылка на скринкаст: https://disk.yandex.ru/i/iu3F_G1cffTh7A

Вывод:

В ходе данной лабораторной работы были изучены и реализованы числовые методы хорд, касательных и делений отрезков для нахождения корней нелинейных уравнений, а также рассмотрено выполнение отделение корней.

Сравнивая результаты работы программы для НУ можно заметить, что метод хорд и касательных точнее, чем метод деления отрезков: чем выше точность, тем ближе к теоретическому результату ответ в методе деления отрезков, а ответы в методах хорд и касательных при любой точности совпадает с теоретическим результатом. Из этого можно сделать вывод, что метод деления отрезков среди рассмотренных методов самый неточный.

Отчет Закаблуковой А.Э.

Лабораторная работа №3

Численные методы решения нелинейных уравнений

Цель: Разработать программный код для отделения корней нелинейных уравнений; разработать программный код для нахождения корней нелинейных уравнений методом хорд, методом касательных, методом деления отрезков. Провести сравнительный анализ результатов.

Используемое оборудование: ПК, Microsoft Office, PyCharm.

Часть 1.

Постановка задачи: Разработать программный модуль, который выполняет отделение корней.

Фрагмент кода для отделения корней:

```
850 #Код окна для отделения корней нелинейного уравнения
851 def otdel_korney():
852     def l_gran(C):
853         i = -1
854         while nu_func(i) > C:
855             i *= 2
856         return i
857     def p_gran(C):
858         i = 1
859         while nu_func(i) < C:
860             i *= 2
861         return i
862     def monoton(a, b, step = 1e-6):
863         x = a
864         sign = np.sign(nu_proixv_func(x))
865         while x < b:
866             if sign == np.sign(nu_proixv_func(x + step)):
867                 x += step
868             else:
869                 return False
870         return True
871
872     def ot_kor(a, b):
873         while True:
874             mon = monoton(a, b, step=1e-6)
875             if (nu_func(a) * nu_func(b) < 0) and (mon == True):
876                 return a, b
877             else:
878                 mid = (b - a) * 0.5
879                 if nu_func(mid) < 0:
880                     a = mid
881                 else:
882                     b = mid
883     return ot_kor(l_gran(0), p_gran(0))
```

Результат:

Уравнение:

$$x^3 + 2x - 6 = 0$$

Отрезок:

Отрезок (-1, 2)

Часть 2.

Постановка задачи: Разработать программный модуль, который находит корень нелинейного уравнения методом хорд.

Фрагмент кода для метода хорд:

```
771     #Код окна решения нелинейных уравнений методом хорд
772     def resh_nu_met_chord():
773         e = float(dane.get())
774         n = float(dann.get())
775         a, b = resh_otdel_korney()
776         iter = 0
777         while abs(nu_func(b)) > e and iter < n:
778             c = (a * nu_func(b) - b * nu_func(a)) / (nu_func(b) - nu_func(a))
779             if nu_func(c) * nu_func(a) < 0:
780                 b = c
781             else: a = c
782             iter += 1
783         print(f'Корень нелинейного уравнения с точностью {e} и {n} разбиений равен {c}')
```

Часть 3.

Постановка задачи: Разработать программный модуль, который находит корень нелинейного уравнения методом касательных.

Фрагмент кода для метода касательных:

```
836     #Код окна решения нелинейных уравнений методом касательных
837     def resh_nu_met_kasat():
838         e = float(dane.get())
839         a, b = resh_otdel_korney()
840         if nu_func(a) * nu_func(b) > 0:
841             x = a
842         else: x = b
843         print(f'При начальном значении x = {x}')
844         x1 = x - nu_func(x) / nu_proixv_func(x)
845         while abs(x1-x) >= e:
846             x = x1 - nu_func(x1) / nu_proixv_func(x1)
847             x1 = x - nu_func(x) / nu_proixv_func(x)
848         print(f'Корень нелинейного уравнения с точностью {e} равен {x}')
849
```

Часть 4.

Постановка задачи: Разработать программный модуль, который находит корень нелинейного уравнения методом деления отрезков.

Фрагмент кода для метода деления отрезков:

```

805 #Код окна решения нелинейных уравнений методом деления отрезков
806 def resh_nu_met_del_otr():
807     e = float(dane.get())
808     a, b = resh_otdel_korney()
809     while (b-a) > e:
810         del_pol = (a+b)/2
811         if nu_func(a) * nu_func(del_pol) < 0:
812             b = del_pol
813         else: a = del_pol
814     print(f'Корень нелинейного уравнения с точностью {e} равен {(a+b)/2}')
815

```

Таблица результатов:

Уравнение:

$$x^3 + 2x - 6 = 0$$

Отрезок (-1, 2)

Метод	e=0,001	e = 0,0001	e=0,00001	e=0,000001	Точное значение
Хорд	1.456164	1.456164	1.456164	1.456164	1.456164
Касательных	1.456164	1.456164	1.456164	1.456164	1.456164
Деления отрезков	1.456176	1.456130	1.456162	1.456164	1.456164

Файл с программой: <https://disk.yandex.ru/d/QxrSTxIwro7Vcg>

Ссылка на скринкаст: https://disk.yandex.ru/i/iu3F_G1cffTh7A

Вывод:

В ходе данной лабораторной работы были изучены и реализованы числовые методы хорд, касательных и делений отрезков для нахождения корней нелинейных уравнений, а также рассмотрено выполнение отделение корней.

Сравнивая результаты работы программы для НУ можно заметить, что метод хорд и касательных точнее, чем метод деления отрезков: чем выше точность, тем ближе к теоретическому результату ответ в методе деления отрезков, а ответы в методах хорд и касательных при любой точности совпадает с теоретическим результатом. Из этого можно сделать вывод, что метод деления отрезков среди рассмотренных методов самый неточный.